Data structure

Time Complexity cont.,

Constant time:

```
Sum;
Sum=sum+i;   // constant time 1
```

Time complexity: O(1)

Loop: executes n time

```
Sum;
For (i=1; i<=n; i++)          //n
Sum=sum+i;   // constant time 1
```

Time complexity: 1+n = O(n)

Nested Loop:

```
For (i=1; i<=n; i++)  {          //Outer loop executes n times
For (j=1; j<=n; j++)
//statement
}                                //Inner loop executes n times
```

} Time complexity: n *n = $O(n^2)$

Constant time

*, /, %, +, -

++, --

+=, -=,/=, *=

Examples for complexity

Example 1

- X= 1; 1
- Y= 2; 1
- Z= x +y ; 1
- Print z;

- The complexity = 1+1+1 = 3

Example 2

- X= 20 1
- If x< = 10 1

  print x;

Complexity = 1 +1 =2

Example 3

- Sum;
- For (i=1;i<=n,i++) n
- Sum = sum+i; 1

- Time complexity 1+n
- By using big order (take the biggest degree+ cancel the constatnt)

→ 1+ n → O(n)

Example 4

- Sum = n*(n+1)/2, 1

- Time complexity = 1

- If n = 5 then sum = 5*(5+1)/2= 30/2=15
- 1+2+3+4+5 = 15

Example 5

```
Int i;
i=0;                                  1
For (i=0;i<n,i++) {                   n
Print I;}
```

Complexity time = 1+n = O(n)

Example for nested loop

```
Int i;
i=0;                                  1
For (i=0;i<n,i++) {                   n
Print I;}
For(j=0;j<n;j++)                      n
For (int k=0;k<n;k++)                 n
Print j+k;
```

Nested loop

Complexity time = 1 + n + n*n = $1+n+n^2$ → $O(n^2)$

Example for log n

```
Int i;
i=1;                          1
For (i=0;i<n,i*2)             log n
Print I;
```

Notice that
For loop add or subtract → n
Multiply or division → log n

Complexity time = 1 + log n → O(log n)

Example for log n

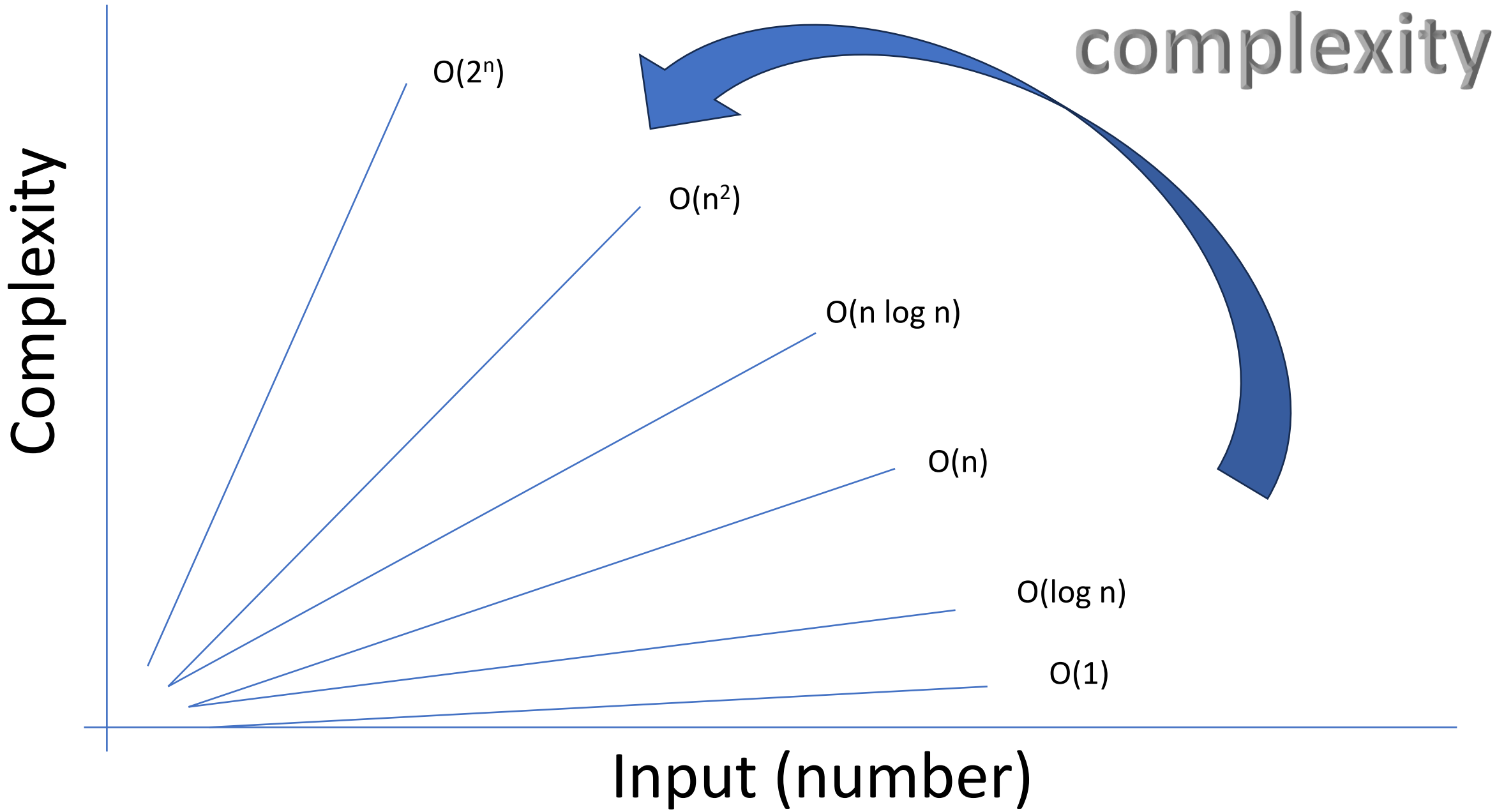
```
Int I,j;
i=j=0;                          1
For (i;i<n,i++)                 n
For (j;j<n,j=j/3)               log n
Print I + j;
```

Complexity time = 1 + n * log n → O(n log n)

Example for nested loop

```
Int i;
i=0;                                  1
For (i=0;i<n,i++)                 n
Print I;
For(j=0;j<n; j=j/3)                   log n
Print j;
```

Complexity time = 1 + n + log n → O(n)


complexity
Complexity
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n)
O(1)
Input (number)

Finding complexity

Generally, we have 6 cases

1. Statements
2. If else
3. Loop
4. Nested loop
5. Function call

Statements

```
statement 1;
statement 2;
...
statement k;
```

The total time is found by adding the times for all statements:

total time = time(statement 1) + time(statement 2) + ... + time(statement k)

If each statement is "simple" (only involves basic operations) then the time for each statement is constant and the total time is also constant: O(1).

If else

```
if (cond) then
block 1 (statements)
else
block 2 (statements)
end if;
```

Here, either block 1 will execute, or block 2 will execute Therefore, the worst-case time is the slower of the two possibilities:

max(time(block 1), time(block 2))

If block 1 takes O(1) and block 2 takes O(N), the if-then else statement would be O(N)

Loop

```
for I in 1 .. N loop
sequence of statements
end loop
```

The loop executes N times, so the sequence of statements also executes N times.

If we assume the statements are O(1), the total time for the for loop is N * O(1), which is O(N) overall.

Nested loop

```
for I in 1 .. N loop
    for J in 1 .. M loop
        sequence of statements
    end loop;
end loop;
```

The statements in the inner loop execute a total of N * M times. Thus, the complexity is O(N * M).

Functions call

The behavior of function is same as statement if called once

Its behavior is statement in loop if it is called in loop

Its behavior is more like nested loop if it is called inside loop and it has an characteristic loop inside as well

Sheet 1

Consecutive Statements:

```
int x=2;
i=1;
x=x+1;
```

Total time = 3 units

```
for (i=1; i<=n; i++)  {          //loop executes n times
            //statement
}
```

```
for (i=1; i<=n; i++)  {
            for (j=1; j<=n; j++)  {
            //statement
            }
}
```

Total time = n^2 units

Time complexity: $n^2+n+3 = O(n^2)$

If then else Statements:

```
if (n==0)  {
            //statement
}

else {
            for (j=1; j<=n; j++)
                        //statement

}
```

For if part:

n==0 takes constant time

Statement inside if takes constant time

Total time:1+1 =O(1)

For else part:

n==0 takes constant time

Statement get executed n times

Total time:1+n =O(n)

Time complexity: Worst case running time = **Test + if part or else part** (whichever is larger) = 1+n = O(n)

```
for (i=1; i<=n; i=i*2){
        //statement
}
```

Time complexity: $O(\log_2 n)$

$\log_2 (8)=3$

The above logarithm says " how many times, 2 has been multiplied by itself in order to obtain value 8"

Iter 1	Initially	$i=1$	2^0
Iter 2		$i=2$	2^1
Iter 3		$i=4$	2^2
..			
Iter k		$i=n$	2^{k-1}

$n = 2^{k-1}$

$K-1 = \log_2 n$

$K=\log_2 n +1$

```
for (i=n; i>=1; i=i/2){
        //statement
}
```

Time complexity: $O(\log_2 n)$

Iter 1	$i= n/ 2^0$
Iter 2	$i= n/ 2^1$
Iter 3	$i= n/2^2$
..	
Iter k	$i=n/2^{k-1} = 1$

$n/2^{k-1} = 1$

$n = 2^{k-1}$

$K-1 = \log_2 n$

$K=\log_2 n +1$

```
Int i;
i=1;                                  // 1
For (i=1; i<=n; i=i*2)                //log n
Print i;
```

Time complexity: $1+\log n = O(\log_2 n)$

```
For (int i=0; i<n; i++)               //n
For (int j=0; j<n; j++)               //n
For (int k=0; k<n; k=k*2)                 // log n
Print i+j+k;
```

Time complexity: $n*n*\log n = n^2 \log n = O(n^2 \log_2 n)$

```
Int i, j;
i=j=0;                                //1
For (int i=0; i<n; i++)               //n
For (int j=0; j<n; j=j/3)               //log n
Print i+j;
```

Time complexity: $1 + n*\log n = O(n \log_3 n)$

```
Int i, j;
i=j=0;                          //1
For (int i=0; i<n; i++)         //n
For (int j=0; j<n; j=j/3)       //log n
Print i+j;
```

Time complexity: $1 + n*\log n = O(n \log_3 n)$

```
For (int i=n/2; i<n; i++)       //n/2
For (int k=0; k<n; k=k*2)       // log n
For (int j=0; j<n; j=2*j)       // log n
Print i+j+k;
```

Time complexity: $n/2*\log n*\log n = n* \log n* \log n = O(n (\log_2 n)^2)$